

Webamc 1.0.0 — A web interface for latex AMC MCQ

Sami Evangelista

April 8, 2026

Abstract

This document describes the installation and use of the webamc python package that allows to submit and view latex AMC MCQ through a web interface. It consists of:

- a command-line tool allowing to compile latex MCQ ;
- and a web interface allowing to load compiled MCQ in a database and view them.

Contents

1	Package installation	3
1.1	Dependencies installation	3
1.2	Webamc installation	3
2	Database and Web server configuration	4
2.1	Database creation	4
2.2	Database initialisation	4
2.2.1	Tables creation	4
2.2.2	CSV files loading	4
2.3	Web server execution and configuration	5
2.4	Apache configuration	5
3	File compilation and submission	6
3.1	Structuring tex files for webamc	6
3.1.1	Header tex file - <code>_header.tex</code>	7
3.1.2	Exercise tex file - <code>_exercise.tex</code>	7
3.1.3	MCQ tex file - <code>_mcq.tex</code>	7
3.1.4	Webamc metadata file - <code>_webamc.tex</code>	7
3.1.5	Question pack file - <code>_pack.json</code>	7
3.1.6	Question file - <code>qst-XXX.tex</code>	7
3.2	Basics on compilation	8
3.3	Webamc metadata	8
3.3.1	Attribute list	8
3.3.2	Attribute validity	9
3.4	Component compilation and submission	9
3.5	JSON file format for question packs	9
3.6	Compilation configuration	11
4	Configuration files	13
5	CSV files	17
5.1	Table <code>usr</code>	17
5.2	Table <code>local_auth</code>	17
5.3	Table <code>cas_auth</code>	17
5.4	Table <code>tag</code>	17
5.5	Table <code>tbl</code>	17
5.6	Table <code>admin</code>	17

5.7	Table grp	18
5.8	Table usr_grp	18
6	Authors	19
7	Credits	20

1 Package installation

This section describes the installation procedure for the webamc package. This procedure has been realised on *Debian 13*.

1.1 Dependencies installation

We will first update the system.

```
$ sudo apt update
$ sudo apt upgrade
```

For a basic use of webamc you will need a python3 environment and optionally the git system (if you wish to install webamc from the development version).

```
$ sudo apt install python3 python3-pip python3-venv git
```

If you install webamc on the web server side you will need the apache web server, the postgresql database manager, and the wsgi module so that our web server can execute our python scripts.

```
$ sudo apt install apache2 postgresql libapache2-mod-wsgi-py3 libpq-dev
```

Now if you install webamc on the client side (i.e., to compile latex MCQ that you intend to load to the web interface) you will need a tex system able to compile tex files to pdf files:

```
$ sudo apt install texlive-latex-base texlive-extra-utils
```

1.2 Webamc installation

We will run python in a virtual environment `py3-venv` that we create here in `/home/webamc`:

```
$ whoami
webamc
$ cd
$ pwd
/home/webamc
$ python3 -m venv py3-venv
```

We then activate the virtual environment.

```
$ source py3-venv/bin/activate
```

We can finally install webamc either from the repository:

```
(py3-venv) $ git clone https://depot.lipn.univ-paris13.fr/evangelista/webamc.git
(py3-venv) $ cd webamc
(py3-venv) $ pip3 install .
```

or from the last available release:

```
(py3-venv) $ pip3 install https://lipn.univ-paris13.fr/evangelista/webamc/dl.php?release=latest
```

2 Database and Web server configuration

This section describes the database and web server configuration procedure. It has been realised on a *Debian 13* system and is intended for system administrators. It assumes that you have already installed the webamc package, following the instructions in Section 1.

2.1 Database creation

Once package has been installed, a database must be created. Right now the database cannot be created with the webamc package, so we will interact with the server using the `psql` client tool. First connect to the database server using `postgres` identity.

```
$ sudo -u postgres psql
```

In the `postgres` interpreter, create the database, a database user and grant this user all privileges on the created database:

```
create user webamc_user password 'webamc_password';
create database webamc;
alter database webamc owner to webamc_user;
grant all privileges on database webamc to webamc_user;
```

2.2 Database initialisation

The next step is to initialise the database: we have to create tables and put some initial data in these. This is done via the `webamc` tool provided by the package.

The configuration of the webamc command-line tool and web interface is done via a JSON file defining several configuration parameters (e.g., database access parameters). An example file can be found in the `examples/cfg` directory of the repository. Section 4 describes the different configuration parameters.

By default, the configuration file is loaded from the `.local/share/webamc/config.json` file in the home directory of the user.

2.2.1 Tables creation

First we need to initialise the database structure and create all its table:

```
$ webamc --config-file /path/to/my/config.json init
```

The command normally silently terminates. An error will likely be due to a misconfiguration of the database access.

2.2.2 CSV files loading

The `webamc loadcsv` command can be used to load records in the database. To use it, simply type this command followed by the path of one (or more) CSV file(s).

```
$ webamc --config-file /path/to/my/config.json loadcsv *.csv
```

The `webamc genexamples` command creates a bunch of example CSV files in the directory passed as argument.

```
$ mkdir csv && webamc genexamples csv
$ ls csv
00-usr.csv  01-cas_auth.csv  02-basic_auth.csv  03-tag.csv
04-grp.csv  05-usr_grp.csv   06-tbl.csv         07-admin.csv
$ webamc --config-file /path/to/my/config.json loadcsv csv/*.csv
```

For a basic setup, you will at least need to insert a user with administration privileges and an authentication mode so that she/he can access the web interface. CSV files can also be loaded from the web interface. Section 5 describes the structure of CSV files.

2.3 Web server execution and configuration

Once the webamc package is installed, you can simply launch webamc's web server via `uvicorn` and bind it to TCP port, e.g., 8008.

```
$ uvicorn webamc.www.app:app --port 8008
```

You can also change set the `CONFIG_FILE` environment variable to specify the configuration file to load:

```
$ CONFIG_FILE=/path/to/my/config.json uvicorn webamc.www.app:app --port 8008
```

The `webamc.service` file in the root directory of the repository contains a minimal service specification to interact with the service via `systemd`.

2.4 Apache configuration

Now that our server listens on the 8008 port, we configure apache to redirect requests addressed to our server to this port. The first step is to enable the `proxy_http` module of apache:

```
$ sudo a2enmod proxy
$ sudo a2enmod proxy_http
```

Then we have to add one line in the configuration of apache's virtual host to pass our server all requests on paths starting by, e.g., `/webamc`.

```
ProxyPass /webamc http://localhost:8008
```

Then restart apache:

```
$ sudo systemctl restart apache2.service
```

3 File compilation and submission

This section describes how the webamc command-line tool can be used to prepare, compile and submit MCQ that can be loaded via the web interface. It assumes that you have already installed the webamc package, following the instructions in Section 1.

3.1 Structuring tex files for webamc

MCQ submitted to the web interface may be of two forms:

- either a list of questions ;
- or a list of exercises, each exercise being a list of questions.

From now on, the term *component* will be used to denote either an MCQ, or an exercise, or a question. In addition to all components that are part of it, an MCQ or an exercise may have a statement, explaining, e.g., the context of all its components. This is the purpose of files `_mcq.tex` and `_exercise.tex` to contain this statements.

The following rules must be respected so that webamc can correctly compile your code:

- The content of an MCQ or an exercise must be placed in a single directory.
- To be recognised as an MCQ directory, the directory must contain an `_mcq.tex` file containing the MCQ statement even if it is empty.
- Likewise, to be recognised as an exercise directory, the directory must contain an `_exercise.tex` file containing the exercise statement even if it is empty.
- Questions must be placed in files of which the name is prefixed by `qst-`.

This is, for example, the directory structure of an MCQ containing five question files:

```
some_mcq
├── _header.tex
├── _mcq.tex
├── qst-aaa.tex
├── qst-bbb.tex
├── qst-ccc.tex
├── qst-ddd.tex
└── qst-eee.tex
```

This is another example of the directory structure of a two exercises MCQ (the first one with 2 question files and the second one with three question files):

```
some_other_mcq
├── _header.tex
├── _mcq.tex
├── exe-fst
│   ├── _exercise.tex
│   ├── qst-fst-1.tex
│   └── qst-fst-2.tex
├── exe-snd
│   ├── _exercise.tex
│   ├── qst-snd-1.tex
│   ├── qst-snd-2.tex
│   └── qst-snd-3.tex
```

More complex examples may be found in the examples `examples/mcq` directory.

You may also submit directories that do not contain any MCQ but only question files. This is useful if you, for example, wish to record some questions in the database before creating question packs (see Section 3.5) that may contain these question.

Note that all the file names listed above (`_header.tex`, `_exercise.tex`, ...) are only default names and may be redefined through configuration files (see Section 3.6 for further details).

3.1.1 Header tex file - `_header.tex`

An `_header.tex` file may contain latex code that will be placed in the preamble of the latex document generated on-the-fly for each component of the directory. Typically, you will write in this file all `\usepackage{...}` instructions required to compile latex files in this directory. A directory inherits from any `_header.tex` contained in any of its ancestor directory. For example, let's consider this directory structure:

```
some_dir
├── _header.tex      (content: \usepackage{packa})
└── another_dir
    ├── _header.tex  (content: \usepackage{packb})
    └── _exercise.tex (content: This is a new exercise)
```

When compiling the exercise file `_exercise.tex`, the latex document compiled will contain the following code:

```
\documentclass{article}
\def\WEBAMC{1}
\usepackage{packa}
\usepackage{packb}
\begin{document}
This is a new exercise
\end{document}
```

3.1.2 Exercise tex file - `_exercise.tex`

An `_exercise.tex` contains the tex code of an exercise preamble, that is, the exercise statement required for the understanding of the questions enclosed in this exercise. The `CODE` attribute must be defined in this file (see Section 3.3).

3.1.3 MCQ tex file - `_mcq.tex`

An `_mcq.tex` contains the tex code of an MCQ preamble, that is the MCQ statement required for the understanding of the components (questions or exercises) enclosed in this MCQ. The `CODE` and `TITLE` attributes must be defined in this file (see Section 3.3).

3.1.4 Webamc metadata file - `_webamc.tex`

The content of this file is never compiled by webamc, but it may be used to set some attributes for all the components enclosed in the directory. For instance, if an MCQ directory contains a `_webamc.tex` file with the single line below:

```
%webamc RND=1
```

then this means that the content of all components of the MCQ (i.e., its exercises, the questions within its exercises, and the answers of all its questions) will be organised randomly.

As for `_header.tex` files, a directory inherits from any `_webamc.tex` contained in any of its ancestor directory.

3.1.5 Question pack file - `_pack.json`

This is a JSON file containing the description of a pack of questions that can be used to form an exercise or an MCQ. Please consult this Section 3.5 for further help on the JSON file format.

3.1.6 Question file - `qst-XXX.tex`

Any tex file which name starts with prefix `qst-` is considered by webamc as a question file, that is, a file containing questions. A question file may contain any tex code, but only the code enclosed within questions will be compiled by webamc, e.g., tex code between questions will be ignored.

3.2 Basics on compilation

Right now, the compilation of the tex files that are part of your MCQ is done on the client side using the `webamc` command line tool. This has the advantage to alleviate the server but, since tex files are compiled once before submitting the MCQ to the web interface, this prevents MCQ from being dynamic.

When analysing your tex files, each question and each answer of your MCQ will be compiled separately in order to generate a separate png file. For example, let's say your MCQ contains the following question:

```
\begin{questionmult}{math-sqrt}
  $\sqrt{81}$ = ?
  \begin{reponses}
    \bonne{9}
    \bonne{$3^2$}
    \mauvaise{19}
    \mauvaise{6}
    \mauvaise{8}
  \end{reponses}
\end{questionmult}
```

Then `webamc` will on-the-fly generate and compile 6 tex documents (1 for the question statement and 1 for each answer) to produce 6 png files. As a consequence, question and answer statements must be fully context independent (e.g., with no `\label{...}` in it).

When compiling a latex file, `webamc` will define in the document's preamble a macro called `WEBAMC` as follows:

```
\def\WEBAMC{1}
```

This may be useful if you want to write latex code specific for the web version of your MCQ, for example:

```
\ifdefined\WEBAMC
(Nota: This is the web version of ...)
\fi
```

It is also very important to note that during compilation, all files are compiled from the directory in which the `_mcq.tex` file is located. Therefore all paths that may appear in your question files (e.g., if you include graphics in your questions) must be relative to that directory.

However, if you compile a directory that does not contain an `_mcq.tex` file but only, e.g., question files, each question file is then compiled from the directory it is located in.

3.3 Webamc metadata

Several component attributes can be set directly in tex files with specific comments. `Webamc` will consider as an attribute definition any comment that starts at column one and that follows this pattern:

```
%webamc ATTRIBUTE_NAME=ATTRIBUTE_VALUE
```

Note that once loaded in the web interface all these attributes (except the `CODE`) can be modified through the web interface.

3.3.1 Attribute list

Attribute `CODE` This defines the code of the component. Codes must be unique and not null. Therefore this attribute is mandatory in exercise tex files (`_exercise.tex`) and MCQ tex files (`_mcq.tex`). For a question, the code is directly extracted from the tex code (i.e., `id-qst` in `\begin{question}{id-qst}`).

Attribute `DIFFICULTY` This defines the difficulty of the component. It is an integer ranging from 1 to 5. This is useful if you want to create question packs based on their difficulty (see this Section 3.5).

Attribute `RND` This defines whether or not the component content (e.g., questions of an exercise or answers of a question) is randomized. Accepted values are 0 and 1. Default value is 0.

Attribute STANDALONE Only valid for questions (for now), this defines whether or not the question is a standalone question that can be selected in question packs. Non standalone questions are typically context dependent questions (e.g., questions based on an exercise statement). Accepted values are 0 and 1. Questions that are not part of an exercise or an MCQ are always considered as standalone questions. Otherwise, the default value is 0.

Attribute TAG This adds a tag to the component. Several tags can be added through repeated assignments to that attribute. This is useful if you want to create question packs based on their tags (see this Section 3.5).

Attribute TITLE This defines the title of the component that will appear in the web interface. Note that any tex macro appearing in it will be ignored, i.e., not compiled, and will appear as such in the web interface. This attribute is mandatory in MCQ tex files (`_mcq.tex`).

Attribute VISIBLE This defines the visibility of the component. A non visible component will not appear in the web interface, except for the user that submitted it. Default value is 1.

3.3.2 Attribute validity

Some attributes are mandatory in some files whereas some others may be meaningless in others. The table below synthesises attribute availability in the different tex files analysed by webamc. In this table:

- 0 means that, if present in the file, the attribute will be ignored ;
- 1 means that the attribute will be considered in the file although it is not mandatory ;
- and 2 means that the attribute is mandatory in the file.

	<code>_exercise.tex</code>	<code>_header.tex</code>	<code>_mcq.tex</code>	<code>_webamc.tex</code>	<code>qst-XXX.tex</code>
CODE	2	0	2	0	0
DIFFICULTY	1	0	1	1	1
RND	1	0	1	1	1
STANDALONE	0	0	0	1	1
TAG	1	0	1	1	1
TITLE	1	0	2	0	0
VISIBLE	1	0	1	1	1

3.4 Component compilation and submission

The `webamc compile` command takes two arguments:

- a directory containing some component(s) to compile ;
- and the name of the zip archive (without the `.zip` suffix) that you will get in output.

You will find several examples in the `examples` directory of the package. Note that this process may take a while (typically a few minutes for large MCQ) since, as said above, webamc will compile each component and each answer separately. The zip archive you get in output can be submitted through the web interface.

3.5 JSON file format for question packs

When present in an exercise or an MCQ directory, a `_pack.json` file specifies that this one is composed of a set of standalone questions (i.e., questions for which the `STANDALONE` attribute has been set to 1). The purpose of the `_pack.json` is to specify how to pick these questions. It consists basically of a sequence of operations successively applied. Available operations are:

- `all`: select all available questions
- `sort`: sort questions (by `difficulty` or by `code`)
- `shuf`: select questions randomly
- `head`: select the first questions from those selected so far

- **with-code**: select questions based on their codes
- **with-difficulty**: select questions based on their difficulty
- **with-tag**: select questions based on their tags

An operation is specified with a dictionary having the following keys:

- **op**: the operation realised, among those listed above
- **arg**: for some operations, specify an argument (e.g., the number of questions selected for operation **head**)
- **rev**: for **with-XXX** operations, operate a negation during the selection (e.g., to select questions that do *not* have a specific tag)
- **content**: questions on which this operation is realised, also specified using an operation dictionary. By default all available questions are taken.

Here are a few examples:

- select all available questions from the database:

```
{
  "op": "all"
}
```

- select all available questions from the database and order them randomly:

```
{
  "op": "shuf"
}
```

- randomly pick 5 questions:

```
{
  "op": "head",
  "arg": 5,
  "content": {
    "op": "shuf"
  }
}
```

- take all questions with tag **réseaux** and with a difficulty of 4 or 5 and shuffle them:

```
{
  "op": "shuf",
  "content": {
    "op": "with-difficulty",
    "arg": [4, 5],
    "content": {
      "op": "with-tag",
      "arg": ["linux"]
    }
  }
}
```

- take all questions that do not have the **linux** tag:

```
{
  "op": "with-tag",
  "rev": true,
  "arg": ["linux"]
}
```

- take all questions having the `réseaux` tag and sort them by difficulty:

```
{
  "op": "sort",
  "arg": "difficulty",
  "content": {
    "op": "with-tag",
    "arg": ["réseaux"]
  }
}
```

It is also possible to realise the union of several selections using lists of operations. For example, this specifies a question pack with all questions having the `linux` tag or with a difficulty level strictly less than 3 ordered randomly.

```
{
  "op": "shuf",
  "content": [
    {
      "op": "with-tag",
      "arg": ["linux"]
    },
    {
      "op": "with-difficulty",
      "rev": true,
      "arg": [3, 4, 5]
    }
  ]
}
```

Last, note that the web interface also provides a form to directly create question packs rather than specifying them with a JSON file.

3.6 Compilation configuration

Several configuration parameters alter the compilation process. Therefore after installation you may wish to check your configuration file, although the default values should work fine. Section 4 also gives the default configuration.

These parameters configure the way the tex-to-pdf compiler is invoked:

- `tex2pdf_exe`: name of the executable (that can be found in your `PATH` environment variable), or absolute path of the executable that will be used to compile tex files to pdf files
- `tex2pdf_exe_args`: argument list given to this executable. The two following strings *must* appear in this list:
 - `{out_dir}`: placeholder for the directory name in which the tex-to-pdf compiler will store output file
 - `{tex_file}`: placeholder for the input tex file
- `log_file`: path of the log file in which webamc will store compilation results

You may also change the name of the default file names that webamc will handle (as described in Section 3.1) by changing these configuration parameters:

- `json_file_pack`
- `tex_file_exercise`
- `tex_file_header`
- `tex_file_mcq`
- `tex_file_question_prefix`
- `tex_file_webamc`

You can also specify which tex macros or environments will be handled by webamc with these configuration parameters:

- `tex_envs_choices`
- `tex_envs_question`
- `tex_envs_question_mult`
- `tex_macros_choice`
- `tex_macros_choice_correct`
- `tex_macros_lastchoices`

4 Configuration files

This section lists the different configuration parameters of a webamc configuration. By default, the configuration is stored in:

- `$HOME/.local/share/webamc/config.json` on unix like systems
- `$HOME/AppData/Roaming/webamc` on windows systems

Parameters may alter either the web interface ; or the command-line interface (cli) tool ; or both. This is marked with a checkmark or a cross in the list below.

- `auth_account_creation_enabled` cli: ✗ web: ✓
 - Description: true if account creation is allowed via the web UI
 - Type: `bool`
 - Default value: `true`
- `auth_cas_enabled` cli: ✗ web: ✓
 - Description: true if authentication through the CAS server is enabled
 - Type: `bool`
 - Default value: `true`
- `auth_local_enabled` cli: ✗ web: ✓
 - Description: true if local system authentication is enabled
 - Type: `bool`
 - Default value: `true`
- `base_url` cli: ✗ web: ✓
 - Description: base URL of your web site
 - Type: `string`
 - Default value: `"https://www.example.com"`
- `cas_server` cli: ✗ web: ✓
 - Description: URL of the CAS server
 - Type: `string`
 - Default value: `"https://cas.example.com"`
- `cas_server_name` cli: ✗ web: ✓
 - Description: a description of the CAS server (e.g. Arkham university CAS server)
 - Type: `string`
 - Default value: `"CAS example server"`
- `cas_version` cli: ✗ web: ✓
 - Description: version of the CAS server (2 or 3)
 - Type: `int`
 - Default value: `2`
- `date_format` cli: ✗ web: ✓
 - Description: date format (null for default format) in the babel package syntax
 - Type: `null | string`
 - Default value: `null`
- `date_locale` cli: ✗ web: ✓
 - Description: date locale in the babel package syntax
 - Type: `string`
 - Default value: `"fr"`
- `datetime_format` cli: ✗ web: ✓
 - Description: datetime format (null for default format) in the babel package syntax
 - Type: `null | string`
 - Default value: `null`
- `db_host` cli: ✓ web: ✓
 - Description: name/IP of the host hosting the database server
 - Type: `string`
 - Default value: `"localhost"`
- `db_name` cli: ✓ web: ✓
 - Description: name of the database
 - Type: `string`

- Default value: "webamc"
- **db_password** cli: ✓ web: ✓
 - Description: password of the database user
 - Type: `string`
 - Default value: "webamc_password"
- **db_port** cli: ✓ web: ✓
 - Description: network port used by the database server
 - Type: `null | int`
 - Default value: 5432
- **db_user** cli: ✗ web: ✓
 - Description: database user
 - Type: `string`
 - Default value: "webamc_user"
- **debug** cli: ✗ web: ✓
 - Description: if true exception tracebacks will be displayed in web browser
 - Type: `bool`
 - Default value: `true`
- **icon_size** cli: ✗ web: ✓
 - Description: size of icons in the web interface (24, 32, 48, or 64)
 - Type: `int`
 - Default value: 24
- **inbox_dir** cli: ✗ web: ✓
 - Description: directory in which user PDF annotated sheets will be stored
 - Type: `string`
 - Default value: "/path/to/inbox/dir"
- **json_file_pack** cli: ✓ web: ✗
 - Description: json file of a question pack in an MCQ or exercise directory
 - Type: `string`
 - Default value: "_pack.json"
- **key_qst_next** cli: ✗ web: ✓
 - Description: key used in the web interface to move to the next question
 - Type: `string`
 - Default value: "d"
- **key_qst_prev** cli: ✗ web: ✓
 - Description: key used in the web interface to move to the previous question
 - Type: `string`
 - Default value: "q"
- **key_switch_mode** cli: ✗ web: ✓
 - Description: key used in the web interface for switching fillin mode
 - Type: `string`
 - Default value: "m"
- **lang** cli: ✗ web: ✓
 - Description: language of the web interface
 - Type: `"en" | "fr"`
 - Default value: "fr"
- **log_file** cli: ✓ web: ✗
 - Description: path of the log file used to store compilation result
 - Type: `null | string`
 - Default value: `null`
- **mails_dir** cli: ✗ web: ✓
 - Description: directory in which mails are stored
 - Type: `null | string`
 - Default value: `null`
- **password_ticket_validity** cli: ✗ web: ✓
 - Description: lifespan of a password regeneration ticket, in seconds
 - Type: `int`

- Default value: 3600
- **projects_dir** cli: ✗ web: ✓
 - Description: directory in which user projects will be stored
 - Type: `string`
 - Default value: `"/path/to/projects/dir"`
- **root_path** cli: ✗ web: ✓
 - Description: root path of webamc on the webserver
 - Type: `string`
 - Default value: `"/webamc"`
- **secret_key** cli: ✗ web: ✓
 - Description: secret key of webamc service
 - Type: `string`
 - Default value: `"abcdefghijklmnopqrstuvwxyz0123456789"`
- **service_name** cli: ✗ web: ✓
 - Description: human readable name of the service that will appear in the UI
 - Type: `string`
 - Default value: `"Webamc"`
- **smtp_auth** cli: ✗ web: ✓
 - Description: true if the SMTP server requires authentication
 - Type: `bool`
 - Default value: `true`
- **smtp_eaddr_from** cli: ✗ web: ✓
 - Description: email address that will be appear in the Form field of mails sent
 - Type: `string`
 - Default value: `"no-reply@example.com"`
- **smtp_host** cli: ✗ web: ✓
 - Description: name/IP of the host hosting the SMTP server
 - Type: `string`
 - Default value: `"mail.example.com"`
- **smtp_password** cli: ✗ web: ✓
 - Description: password used to connect to the SMTP server
 - Type: `string`
 - Default value: `"smtp_password"`
- **smtp_port** cli: ✗ web: ✓
 - Description: port the SMTP server listens to
 - Type: `int`
 - Default value: `465`
- **smtp_user** cli: ✗ web: ✓
 - Description: user account used to connect to the SMTP server
 - Type: `string`
 - Default value: `"smtp_user"`
- **tex2pdf_exe** cli: ✓ web: ✗
 - Description: name or absolute path of the tex-to-pdf compiler
 - Type: `string`
 - Default value: `"pdflatex"`
- **tex2pdf_exe_args** cli: ✓ web: ✗
 - Description: list of arguments that must be passed to tex2pdf_exe
 - Type: `list of (string)`
 - Default value: `["-output-directory={out_dir}", "{tex_file}"]`
- **tex_envs_choices** cli: ✓ web: ✗
 - Description: tex environments for choice lists
 - Type: `list of (string)`
 - Default value: `["choices", "choiceshoriz", "reponses", "reponseshoriz"]`
- **tex_envs_question** cli: ✓ web: ✗
 - Description: tex environments for questions
 - Type: `list of (string)`

- Default value: ["question", "questionmult"]
- `tex_envs_question_mult` cli: ✓ web: ✗
 - Description: tex environments for question with multiple correct choices
 - Type: list of (string)
 - Default value: ["questionmult"]
- `tex_file_exercise` cli: ✓ web: ✗
 - Description: tex file of an exercise directory
 - Type: string
 - Default value: "_exercise.tex"
- `tex_file_header` cli: ✓ web: ✗
 - Description: tex file with headers to be used for all tex files of a directory
 - Type: string
 - Default value: "_header.tex"
- `tex_file_mcq` cli: ✓ web: ✗
 - Description: tex file of an MCQ directory
 - Type: string
 - Default value: "_mcq.tex"
- `tex_file_question_prefix` cli: ✓ web: ✗
 - Description: prefix (possibly empty) of tex files containing questions
 - Type: string
 - Default value: "qst-"
- `tex_file_webamc` cli: ✓ web: ✗
 - Description: tex file in a directory containing webamc meta-data
 - Type: string
 - Default value: "_webamc.tex"
- `tex_macros_choice` cli: ✓ web: ✗
 - Description: tex macros for choices
 - Type: list of (string)
 - Default value: ["bonne", "correctchoice", "mauvaise", "wrongchoice"]
- `tex_macros_choice_correct` cli: ✓ web: ✗
 - Description: tex macros for correct choices
 - Type: list of (string)
 - Default value: ["bonne", "correctchoice"]
- `tex_macros_lastchoices` cli: ✓ web: ✗
 - Description: tex macros for last choices
 - Type: list of (string)
 - Default value: ["alafin", "lastchoices"]
- `ticket_length` cli: ✗ web: ✓
 - Description: length of tickets (e.g., password change ticket) generated
 - Type: int
 - Default value: 64

5 CSV files

This section describes the structure of database tables that can be directly fed via CSV files loaded through the web interface or via the `webamc loadcsv` command.

5.1 Table `usr`

- Description: This table defines users.
- Columns:
 - `usr_code` — unique user code
 - `usr_eaddr` — unique electronic address
 - `usr_fst_name` — first name
 - `usr_name` — name

5.2 Table `local_auth`

- Description: This table controls local system authentication. Each row gives access to one user. The login used by the user to authenticate is her/his code. An empty password prevents the user from logging meaning that he/she has to first reset his/her password. If not empty, the password is given plain.
- Columns:
 - `loc_login` — login of the user
 - `loc_usr` — user which is granted access (references `usr.usr_code`)
 - `loc_enabled` — 1 if local authentication is enabled for the user, 0 otherwise
 - `loc_password` — plain-text password

5.3 Table `cas_auth`

- Description: This table controls CAS authentication. Each row gives access to one user.
- Columns:
 - `cas_usr` — user which is granted access (references `usr.usr_code`)
 - `cas_enabled` — 1 is CAS authentication enabled for the user, 0 otherwise
 - `cas_login` — login of the user on the CAS server

5.4 Table `tag`

- Description: This table defines tags that can be associated to MCQ items (e.g., questions).
- Columns:
 - `tag_name` — unique tag name
 - `tag_desc` — tag description (possibly empty)
 - `tag_color` — tag color of the form `#RRGGBB` (possibly empty)

5.5 Table `tbl`

- Description: This table contains the list of names of tables which can be administrated through the web interface, i.e., for which CSV file submission is allowed.
- Columns:
 - `tbl_name` — unique table name

5.6 Table `admin`

- Description: This table defines grants administration privileges to users on tables.
- Columns:
 - `adm_usr` — user which is granted admin right (references `usr.usr_code`)
 - `adm_tbl` — table on which this right admin right is granted (references `tbl.tbl_name`)

5.7 Table *grp*

- Description: This table defines user groups.
- Columns:
 - `grp_name` — unique group name
 - `grp_parent` — parent group name (possibly empty, references `grp.grp_name`)

5.8 Table *usr_grp*

- Description: This table defines group memberships.
- Columns:
 - `ugp_usr` — user (references `usr.usr_code`)
 - `ugp_grp` — group (references `grp.grp_name`)
 - `ugp_right` — right given to the user on the group: 1 for view (right to view MCQ submitted for that group), 2 for submit (right to submit MCQ for that group)

6 Authors

The following people contributed to the development of webamc:

Since 2023 — Sami EVANGELISTA

2024 — Fadela Manel BELAIDOUNI

2024 — Lasry BESKIWIN

2024 — Anas HADRI

2023 — Loic LI

7 Credits

Colton Griffith (<https://fluenticons.co>) designed all icons used in the web UI. Many thanks to him !